

Chapitre 4 : Classes, Objets et Encapsulation

4.1. Concept de Classe et d'Objet

Une **classe** est un plan (ou moule) qui définit des attributs (données) et des méthodes (comportements). Un **objet** est une instance concrète créée à partir de cette classe.

- **`__init__`** : Le constructeur exécuté automatiquement lors de l'instanciation.
- **`self`** : Représente l'instance courante sur laquelle la méthode est appelée.

```
class Voiture:
    # Attribut de classe (partagé par toutes les instances)
    roues: int = 4

    def __init__(self, marque: str, modele: str):
        # Attributs d'instance (propres à chaque objet)
        self.marque: str = marque
        self.modele: str = modele
        self.vitesse: float = 0.0

    def accelerer(self, valeur: float) -> None:
        """Méthode d'instance modifiant l'état de l'objet."""
        self.vitesse += valeur

# Instanciation
ma_voiture = Voiture("Tesla", "Model 3")
ma_voiture.accelerer(50)
print(ma_voiture.vitesse) # Output: 50.0
```

4.2. Encapsulation et Conventions de Masquage

Contrairement à d'autres langages (Java/C++), Python n'a pas de mots-clés stricts comme **`private`** ou **`protected`**. Tout repose sur des conventions d'écriture et le mécanisme du *Name Mangling* :

1. **Public (`attribut`)** : Accessible partout.
2. **Protégé (`_attribut`)** : Indique par convention aux développeurs que l'attribut est interne à la classe et ses sous-classes.
3. **Privé (`__attribut`)** : Déclenche le *Name Mangling* (transformation automatique du nom en `_NomDeLaClasse__attribut`) pour empêcher les accès accidentels depuis l'extérieur.

```
class CompteBancaire:

    def __init__(self, titulaire: str, solde_initial: float):
        self.titulaire = titulaire
        self._numero_compte = "FR76 1234 5678" # Convention protégé
```

```

        self.__solde = solde_initial # Pseudo-privé (Name Mangling)

    def depoter(self, montant: float) -> None:
        if montant > 0:
            self.__solde += montant

compte = CompteBancaire("Jean", 1000.0)
# print(compte.__solde) # AttributeError!
print(compte._CompteBancaire__solde) # Accessible via Name Mangling
(déconseillé)

```

4.3. Décorateurs `@property`, Setters et Deleters

Pour accéder ou modifier des attributs protégés/privés de façon idiomatique sans casser l'interface publique, on utilise le décorateur `@property`.

```

class Produit:

    def __init__(self, nom: str, prix: float):
        self.nom = nom
        self.prix = prix # Utilise le setter sous le capot

    @property
    def prix(self) -> float:
        """Getter pour le prix."""
        return self._prix

    @prix.setter
    def prix(self, valeur: float) -> None:
        """Setter avec validation de donnée."""
        if valeur < 0:
            raise ValueError("Le prix ne peut pas être négatif.")
        self._prix = valeur

    @prix.deleter
    def prix(self) -> None:
        """Deleter exécuté avec `del produit.prix`."""
        print("Suppression du prix")
        del self._prix

p = Produit("Clavier", 49.99)
p.prix = 59.99 # Appelle le setter avec validation
# p.prix = -10 # Lève une ValueError

```

4.4. Méthodes de Classe (@classmethod) et Méthodes Statiques (@staticmethod)

Type de Méthode	Premier Argument	Utilisation Principale
Méthode d'instance	<code>self</code>	Manipuler l'état d'un objet spécifique.
Méthode de classe (@classmethod)	<code>cls</code>	Interagir avec la classe elle-même ou créer des constructeurs alternatifs (<i>factory methods</i>).
Méthode statique (@staticmethod)	Aucun (<code>self/cls</code> omis)	Fonction utilitaire isolée n'ayant besoin d'accéder ni à l'instance ni à la classe.

```
class DateUtils:

    def __init__(self, jour: int, mois: int, annee: int):
        self.jour = jour
        self.mois = mois
        self.annee = annee

    @classmethod
    def depuis_chaine(cls, date_str: str) -> "DateUtils":
        """Factory Method : crée une instance depuis un format 'AAAA-MM-JJ'."""
        annee, mois, jour = map(int, date_str.split("-"))
        return cls(jour, mois, annee)

    @staticmethod
    def est_annee_bissextile(annee: int) -> bool:
        """Fonction utilitaire pure sans état."""
        return (annee % 4 == 0 and annee % 100 != 0) or (annee % 400 == 0)

# Usage
d1 = DateUtils.depuis_chaine("2026-03-30")
print(DateUtils.est_annee_bissextile(2026)) # False
```

Chapitre 5 : Héritage, Polymorphisme et Méthodes Magiques

5.1. Héritage Simple et Fonction `super()`

L'héritage permet à une sous-classe d'hériter des attributs et méthodes d'une classe mère tout en redéfinissant (surcharge) ou étendant certains comportements.

```
class Vehicule:

    def __init__(self, marque: str, vitesse_max: float):
        self.marque = marque
```

```

        self.vitesse_max = vitesse_max

    def decrire(self) -> str:
        return f"Véhicule {self.marque} (Max: {self.vitesse_max} km/h)"

class Moto(Vehicule):

    def __init__(self, marque: str, vitesse_max: float, cylindree: int):
        # Appel du constructeur de la classe parente
        super().__init__(marque, vitesse_max)
        self.cylindree = cylindree

    def decrire(self) -> str:
        # Surcharge de méthode
        base = super().decrire()
        return f"{base} - {self.cylindree}cc"

m = Moto("Yamaha", 220, 600)
print(m.decrire()) # Véhicule Yamaha (Max: 220 km/h) - 600cc

```

5.2. Polymorphisme et Duck Typing

Le **Duck Typing** ("*Si ça marche comme un canard et que ça cancanne comme un canard, c'est un canard*") est le principe fondamental du polymorphisme en Python : le type exact d'une instance importe moins que la présence des méthodes ou attributs requis.

```

class Canard:

    def parler(self):
        return "Coin coin !"

class Chien:

    def parler(self):
        return "Wouf !"

def faire_crier(animal):
    # Fonction polymorphe : accepte n'importe quel objet possédant une
    # méthode `parler`
    print(animal.parler())

faire_crier(Canard()) # Coin coin !

```

```
faire_crier(Chien()) # Wouf !
```

5.3. Héritage Multiple et MRO (Method Resolution Order)

Python prend en charge l'héritage multiple. Lorsqu'une classe hérite de plusieurs mères (problème du diamant), Python détermine l'ordre de résolution des méthodes via l'algorithme **C3 Linearization** (accessible avec `Classe.mro()` ou `Classe.__mro__`).

```
class A:

    def ping(self):
        print("A")

class B(A):

    def ping(self):
        print("B")
        super().ping()

class C(A):

    def ping(self):
        print("C")
        super().ping()

class D(B, C):

    def ping(self):
        print("D")
        super().ping()

d = D()
d.ping()
# Ordre d'exécution MRO : D -> B -> C -> A
# Output:
# D
# B
# C
# A
```

5.4. Méthodes Spéciales (*Dunder Methods*)

Les méthodes spéciales (qui commencent et se terminent par un double underscore `__`) permettent d'intégrer vos propres classes au système d'opérateurs natifs de Python (conversion en chaîne, comparaison, opérations arithmétiques, accès par index, etc.).

Les principales Dunder Methods :

- **Représentation :**
 - `__str__(self)` : Représentation lisible destinée à l'utilisateur final (`print()`, `str()`).
 - `__repr__(self)` : Représentation non ambiguë destinée au débogage (`repr()`).
- **Comparaison :**
 - `__eq__(self, other)` : Opérateur de comparaison `==`.
 - `__lt__(self, other)` : Opérateur d'infériorité `<`.
- **Conteneur & Séquence :**
 - `__len__(self)` : Utilisé par `len()`.
 - `__getitem__(self, key)` : Accès par crochets `obj[key]`.
- **Arithmétique :**
 - `__add__(self, other)` : Opérateur d'addition `+`.

```
class Vector2D:

    def __init__(self, x: float, y: float):
        self.x = x
        self.y = y

    def __repr__(self) -> str:
        return f"Vector2D({self.x}, {self.y})"

    def __add__(self, other: "Vector2D") -> "Vector2D":
        if not isinstance(other, Vector2D):
            return NotImplemented
        return Vector2D(self.x + other.x, self.y + other.y)

    def __eq__(self, other: object) -> bool:
        if not isinstance(other, Vector2D):
            return False
        return self.x == other.x and self.y == other.y

v1 = Vector2D(2, 3)
v2 = Vector2D(4, 1)

print(repr(v1))  # Vector2D(2, 3)
print(v1 + v2)  # Vector2D(6, 4)
```

```
print(v1 == Vector2D(2, 3)) # True
```
